LAB DEVOPS

Pipeline CI/CD Desafio RH

Integração contínua para frontend e backend, publicação de imagem Docker e deployment automatizado.

Projeto: Desafio RH
Autora: Ana Blefari
Data: 09 de novembro de 2026

1 Identificação da entrega

Este relatório registra a construção de uma pipeline CI/CD para o projeto Desafio RH. A solução automatiza a validação do frontend React/TypeScript, do backend Spring Boot/Maven, da imagem Docker e dos deployments no Render e na Vercel.

Item	Referência
Repositório	github.com/anablefari/desafio-rh
Branch da implementação	codex/implementa-lab-cicd
Arquivo da pipeline	.github/workflows/lab-cicd.yml
Commit da implementação	22be71e
Commit da substituição do workflow	e58c01d
Revisão	Abrir Pull Request

Escopo entregue

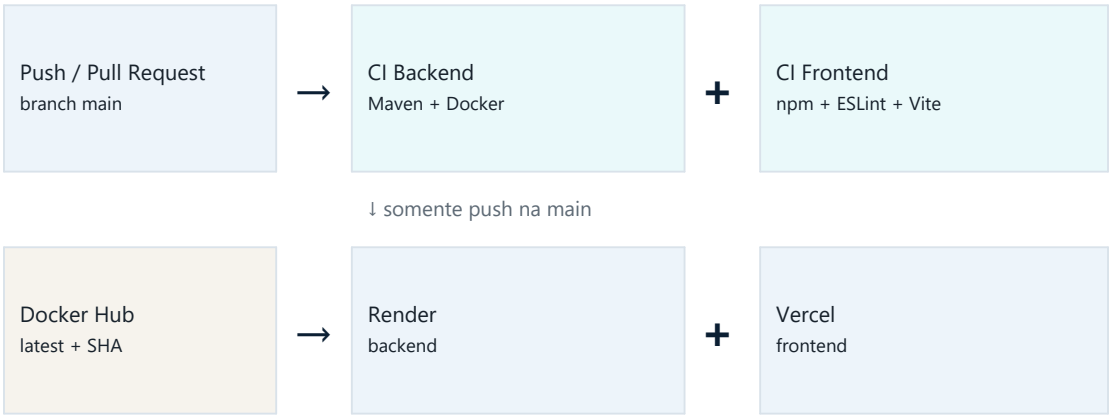
Uma nova pipeline foi criada em arquivo próprio. Ela possui CI independente para backend e frontend, validação Docker e CD condicionado ao sucesso das duas CIs.

Objetivos atendidos

- Executar CI em push para a branch main e em pull requests direcionados à main.
- Realizar checkout, configurar os ambientes e instalar dependências.
- Executar testes, lint, build e validação da imagem Docker.
- Executar CD somente após as duas CIs e apenas em push para a main.
- Publicar frontend e backend em serviços adequados ao projeto.

2 Arquitetura e estratégia de deployment

A estratégia separa validação e publicação. Pull requests executam somente CI. Um push na main habilita o CD, desde que os jobs do frontend e do backend tenham sido aprovados.



Justificativa das ferramentas

Ferramenta	Uso	Justificativa
GitHub Actions	Orquestração	Integração nativa com o repositório e suporte aos eventos push e pull_request.
Docker Hub	Registro de imagens	Armazena a imagem do backend com uma tag estável e outra imutável por commit.
Render	Backend	Executa o serviço Spring Boot a partir da imagem Docker e aceita deploy hook.
Vercel	Frontend	Hospedagem adequada para aplicação React/Vite e deployment automatizado via CLI.

Rastreabilidade do backend

O backend é publicado com as tags latest e o SHA completo do commit. O Render recebe a tag do SHA, vinculando o deployment ao mesmo código que passou pela CI.

3 Estrutura da nova pipeline

Job	Gatilho	Etapas principais	Dependência
ci-backend	Push e PR para main	Java 21, cache Maven, clean verify, Buildx e build Docker linux/amd64.	Nenhuma
ci-frontend	Push e PR para main	Node 24, cache npm, npm ci, ESLint e build Vite.	Nenhuma
cd-backend	Somente push na main	Login no Docker Hub, push latest/SHA e deploy hook do Render.	As duas CIs
cd-frontend	Somente push na main	Vercel pull, build de produção e deploy prebuilt.	As duas CIs

Controles adicionados

- Permissão mínima do GITHUB_TOKEN: contents: read.
- Timeouts para impedir jobs bloqueados indefinidamente.
- Concorrência com cancelamento de execuções antigas da mesma referência.
- Validação explícita de variables e secrets antes do deployment.
- Environments separados: production-backend e production-frontend.

Configurações externas

Tipo	Nome	Finalidade
Variable	DOCKERHUB_USERNAME	Usuário do Docker Hub.
Variable	DOCKERHUB_IMAGE	Nome completo da imagem do backend.
Secret	DOCKERHUB_TOKEN	Token de acesso ao Docker Hub.
Secret	RENDER_DEPLOY_HOOK_URL	Aciona o deployment no Render.
Variable	BACKEND_PUBLIC_URL	URL pública usada pelo frontend.
Variable	VERCEL_ORG_ID	Identifica a conta ou organização Vercel.
Variable	VERCEL_PROJECT_ID	Identifica o projeto Vercel.
Secret	VERCEL_TOKEN	Autoriza a publicação na Vercel.

4 Registro das interações com IA

Foram selecionadas 3 interações relevantes para representar a evolução da solução, desde a análise até a revisão final.

Prompt A - Análise e estratégia
Prompt: Analise os PDFs 07 - CI - Integração Contínua, 08 - CI - GitAction_Docker e Lab CICD-Pipeline e examine este projeto full-stack, composto por frontend React com TypeScript e backend Spring Boot com Maven. Identifique os requisitos obrigatórios e proponha uma estratégia de CI/CD com GitHub Actions, Docker Hub, Render e Vercel.
Resultado: Mapeamento dos requisitos e definição da arquitetura CI/CD.
Problema encontrado: O projeto precisava atender duas tecnologias e dois destinos de deployment.
Alteração realizada: Separação da CI em frontend/backend e escolha de Docker Hub, Render e Vercel.

Prompt B - Implementação da pipeline
Prompt: Implemente do zero a pipeline .github/workflows/lab-cicd.yml. Execute CI em push e pull_request para main; configure Java 21, Maven e validação Docker no backend; configure Node.js 24, npm ci, ESLint e build no frontend; e execute CD somente em push na main depois que as duas CIs forem aprovadas. Publique a imagem no Docker Hub com latest e SHA, acione o Render e publique o frontend na Vercel usando apenas Secrets e Variables.
Resultado: Criação dos quatro jobs e encadeamento CI para CD.
Problema encontrado: Era necessário impedir deploy em pull requests e publicar exatamente a imagem testada.
Alteração realizada: Condição de push na main, needs nas duas CIs e tag Docker baseada no SHA.

Prompt C - Revisão e documentação
Prompt: Revise a nova pipeline quanto a sintaxe, segurança, dependências entre jobs e condições de deployment. Adicione permissões mínimas, timeouts, cache, cancelamento de execuções antigas e validação das configurações. Atualize o README, valide o YAML e envie a implementação para a branch do fork, registrando as limitações locais.
Resultado: Revisão estrutural do YAML, documentação e publicação da branch.
Problema encontrado: O ambiente local não possuía todas as versões exigidas pelo projeto.
Alteração realizada: Versões fixadas nos runners, registro das limitações e validação estrutural do workflow.

5 Evolução, problemas e alterações

Etapa	Resultado	Problema	Ajuste realizado
Análise	Estratégia definida.	Projeto full-stack com processos distintos.	Jobs independentes e CD dependente das duas CIs.
Backend	Maven e Docker integrados.	A imagem precisava ser validada antes da publicação.	Build sem push na CI e push somente no CD.
Frontend	Lint e build automatizados.	Build precisa da URL correta da API.	BACKEND_PUBLIC_URL configurada como Variable.
Deployment	Render e Vercel integrados.	Risco de publicar versão diferente da testada.	Tag imutável baseada no SHA do commit.
Validação	YAML interpretado corretamente.	Java, Node e Docker locais incompatíveis/indisponíveis.	Java 21 e Node 24 configurados nos runners; limitação registrada.

Alterações manuais

- Configuração dos serviços e credenciais externas deve ser feita nas contas Docker Hub, Render, Vercel e GitHub.
- Criação dos environments production-backend e production-frontend no GitHub.
- Abertura e merge do Pull Request depois da revisão da CI.
- Captura das telas do GitHub Actions, Docker Hub, Render e Vercel após a primeira execução completa.

Breve análise do processo

A IA acelerou a leitura dos requisitos, a estruturação do workflow e a identificação de riscos, mas as respostas precisaram ser confrontadas com a arquitetura real do projeto. A principal decisão foi separar CI e CD: pull requests validam o código sem acessar credenciais, enquanto o deployment é autorizado somente após um push na main e o sucesso das duas CIs. O uso do SHA como tag da imagem melhora a rastreabilidade e reduz ambiguidades entre código testado e versão publicada.

6 Validação e evidências

As evidências abaixo foram separadas entre comprovações já obtidas e comprovações operacionais que dependem da configuração das credenciais e da primeira execução do workflow.

Evidência	Status	Comprovação
Implementação versionada	CONFIRMADA	Commit 22be71e publicado na branch do fork.
Substituição do workflow	CONFIRMADA	Commit e58c01d e arquivo lab-cicd.yml.
Sintaxe e estrutura YAML	CONFIRMADA	Parser reconheceu os jobs ci-backend, ci-frontend, cd-backend e cd-frontend.
Ausência de alterações locais	CONFIRMADA	Branch local limpa e rastreando origin/codex/implementa-lab-cicd.
CI no GitHub Actions	PENDENTE	Abrir Pull Request e capturar os dois jobs de CI aprovados.
CD completo	PENDENTE	Configurar secrets/variables, fazer merge e capturar Docker Hub, Render e Vercel.

Importante para a entrega final

Este relatório não apresenta uma execução fictícia. Antes do envio ao formulário, devem ser acrescentadas capturas reais do GitHub Actions aprovado e dos deployments concluídos.

Checklist de evidências finais

Conferência	Evidência esperada
☐	Pull Request exibindo ci-backend e ci-frontend aprovados.
☐	Execução na main com cd-backend e cd-frontend aprovados.
☐	Docker Hub mostrando as tags latest e o SHA do commit.
☐	Render mostrando o backend implantado e acessível.
☐	Vercel mostrando o frontend em produção.
☐	Teste funcional do frontend consumindo a API publicada.

Conclusão

A implementação cobre o fluxo CI para frontend e backend, a validação e publicação Docker e o CD automatizado para Render e Vercel. O código está versionado no fork e pronto para revisão. A conclusão operacional da atividade depende da configuração das credenciais externas e da execução documentada no GitHub Actions.

A Pipeline completa - parte 1 de 4

Conteúdo integral de .github/workflows/lab-cicd.yml, incluído como parte da entrega solicitada.

```
001  name: Lab CI/CD - Desafio RH
002
003  on:
004    push:
005      branches: [main]
006    pull_request:
007      branches: [main]
008
009  permissions:
010    contents: read
011
012  concurrency:
013    group: lab-cicd-${{ github.ref }}
014    cancel-in-progress: true
015
016  jobs:
017    ci-backend:
018      name: Backend - testes e imagem Docker
019      runs-on: ubuntu-latest
020      timeout-minutes: 15
021      defaults:
022        run:
023          working-directory: backend
024
025      steps:
026        - name: Checkout do repositório
027          uses: actions/checkout@v7
028
029        - name: Configurar Java 21
030          uses: actions/setup-java@v6
031          with:
032            distribution: temurin
033            java-version: '21'
034            cache: maven
035            cache-dependency-path: backend/pom.xml
036
037        - name: Compilar e executar testes
038          run: bash ./mvnw --batch-mode --no-transfer-progress clean verify
039
040        - name: Configurar Docker Buildx
041          uses: docker/setup-buildx-action@v4
042
043        - name: Validar build da imagem Docker
044          uses: docker/build-push-action@v7
045          with:
```

A Pipeline completa - parte 2 de 4

```
046           context: backend
047           file: backend/Dockerfile
048           platforms: linux/amd64
049           push: false
050           cache-from: type=gha,scope=backend
051           cache-to: type=gha,mode=max,scope=backend
052
053     ci-frontend:
054       name: Frontend - lint e build
055       runs-on: ubuntu-latest
056       timeout-minutes: 10
057       defaults:
058         run:
059           working-directory: frontend
060
061       steps:
062         - name: Checkout do repositório
063           uses: actions/checkout@v7
064
065         - name: Configurar Node.js 24
066           uses: actions/setup-node@v7
067           with:
068             node-version: '24'
069             cache: npm
070             cache-dependency-path: frontend/package-lock.json
071
072         - name: Instalar dependências
073           run: npm ci
074
075         - name: Analisar código com ESLint
076           run: npm run lint
077
078         - name: Compilar frontend
079           run: npm run build
080
081     cd-backend:
082       name: CD Backend - Docker Hub e Render
083       needs: [ci-backend, ci-frontend]
084       if: github.event_name == 'push' && github.ref == 'refs/heads/main'
085       runs-on: ubuntu-latest
086       timeout-minutes: 15
087       environment: production-backend
088       env:
089         DOCKER_IMAGE: ${{ vars.DOCKERHUB_IMAGE }}
090         DOCKERHUB_USERNAME: ${{ vars.DOCKERHUB_USERNAME }}
```

A Pipeline completa - parte 3 de 4

```
091       DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
092       RENDER_DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK_URL }}
093
094     steps:
095       - name: Checkout do repositório
096         uses: actions/checkout@v7
097
098       - name: Validar configuração do backend
099         shell: bash
100         run: |
101           if [ -z "$DOCKER_IMAGE" ] || [ -z "$DOCKERHUB_USERNAME" ] || [ -z "$DOCKERHUB_TOKEN" ] || [ -z "$RENDER_DEPLOY_HOOK_URL" ]; then
102             echo "Configure DOCKERHUB_IMAGE e DOCKERHUB_USERNAME como Variables; DOCKERHUB_TOKEN e RENDER_DEPLOY_HOOK_URL como Secrets."
103             exit 1
104           fi
105
106       - name: Autenticar no Docker Hub
107         uses: docker/login-action@v4
108         with:
109           username: ${{ env.DOCKERHUB_USERNAME }}
110           password: ${{ env.DOCKERHUB_TOKEN }}
111
112       - name: Configurar Docker Buildx
113         uses: docker/setup-buildx-action@v4
114
115       - name: Publicar imagem do backend
116         uses: docker/build-push-action@v7
117         with:
118           context: backend
119           file: backend/Dockerfile
120           platforms: linux/amd64
121           push: true
122           tags: |
123             ${{ env.DOCKER_IMAGE }}:latest
124             ${{ env.DOCKER_IMAGE }}:${{ github.sha }}
125           cache-from: type=gha,scope=backend
126           cache-to: type=gha,mode=max,scope=backend
127
128       - name: Publicar a nova imagem no Render
129         shell: bash
130         run: |
131           curl --fail --silent --show-error \
132             --request POST \
133             --get \
134             --data-urlencode "imgURL=docker.io/${DOCKER_IMAGE}:${GITHUB_SHA}" \
135             "$RENDER_DEPLOY_HOOK_URL"
```

A Pipeline completa - parte 4 de 4

```
136
137   cd-frontend:
138     name: CD Frontend - Vercel
139     needs: [ci-backend, ci-frontend]
140     if: github.event_name == 'push' && github.ref == 'refs/heads/main'
141     runs-on: ubuntu-latest
142     timeout-minutes: 15
143     environment: production-frontend
144     env:
145       VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
146       VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
147       VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
148       VITE_API_URL: ${{ vars.BACKEND_PUBLIC_URL }}
149
150     steps:
151       - name: Checkout do repositório
152         uses: actions/checkout@v7
153
154       - name: Configurar Node.js 24
155         uses: actions/setup-node@v7
156         with:
157           node-version: '24'
158
159       - name: Validar configuração do frontend
160         shell: bash
161         run: |
162           if [ -z "$VERCEL_TOKEN" ] || [ -z "$VERCEL_ORG_ID" ] || [ -z "$VERCEL_PROJECT_ID" ] || [ -z "$VITE_API_URL" ]; then
163             echo "Configure VERCEL_TOKEN como Secret; VERCEL_ORG_ID, VERCEL_PROJECT_ID e BACKEND_PUBLIC_URL como Variables."
164             exit 1
165           fi
166
167       - name: Instalar Vercel CLI
168         run: npm install --global vercel@latest
169
170       # O projeto Vercel usa Root Directory = frontend. A CLI parte da raiz
171       # para evitar que a configuração resulte no caminho frontend/frontend.
172       - name: Carregar configurações de produção
173         run: vercel pull --yes --environment=production --token="$VERCEL_TOKEN"
174
175       - name: Preparar build de produção
176         run: vercel build --prod --token="$VERCEL_TOKEN"
177
178       - name: Publicar frontend
179         run: vercel deploy --prebuilt --prod --token="$VERCEL_TOKEN"
```